Module 3 Introduction to OOPS Programming

1. Introduction to C++

1. What are the key differences between Procedural Programming and Object-Oriented Programming (OOP)?

Feature	Procedural Programming	Object-Oriented Programming (OOP)
Basic Idea	Uses functions	Uses objects
Focus	Logic (steps)	Data + methods
Approach	Top-down	Bottom-up
Data Security	Less secure	More secure
Reusability	Low	High
Example	C	C++, Java

2. List and explain the main advantages of OOP over POP.

Advantages of OOP over POP

1. **Data Security** – OOP hides data using encapsulation, making programs more secure.
2. **Reusability** – Code can be reused using inheritance.
3. **Easy Maintenance** – Programs are easier to update and fix.
4. **Modularity** – Code is divided into small parts (objects), making it organized.
5. **Flexibility** – Supports polymorphism, allowing one function to behave in different ways.

3. Explain the steps involved in setting up a C++ development environment.

Steps to Set Up C++ Development Environment

1. **Install Compiler** – Install a C++ compiler (like GCC or MinGW).
2. **Install IDE/Editor** – Install an IDE such as Code::Blocks, Dev C++, or VS Code.
3. **Configure Compiler** – Link the compiler with the IDE (if needed).
4. **Write Program** – Create and write your C++ code.
5. **Compile Program** – Convert code into executable file.
6. **Run Program** – Execute the program to see output.

4.What are the main input/output operations in C++? Provide examples.

Main Input/Output Operations in C++

Operation	Description	Example
Input (`cin`)	Takes input from user	`cin >> x;`
Output (`cout`)	Displays output to screen	`cout << x;`

New Line (`endl`)	Moves to next line	`cout << "Hello" << endl;`

2. Variables, Data Types, and Operators

1. What are the different data types available in C++? Explain with examples.

Data Types in C++

Type	Description	Example
int	Stores whole numbers	`int a = 10;`
float	Stores decimal numbers	`float b = 5.5;`
double	Stores large decimal values	`double c = 10.12345;`
char	Stores a single character	`char d = 'A';`
bool	Stores true/false	`bool e = true;`
string	Stores text	`string name = "Raj";`

2. Explain the difference between implicit and explicit type conversion in C++.

Feature	Implicit Conversion	Explicit Conversion
Meaning	Done automatically by compiler	Done manually by programmer
Effort	No effort needed	Requires type casting
Safety	Less control	More control
Example	`int a = 5; double b = a;`	`double x = 5.5; int y = (int)x;`

3. What are the different types of operators in C++? Provide examples of each.

Operator Type	Description	Example
Arithmetic	Performs math operations	`a + b`, `a - b`
Relational	Compares values	`a > b`, `a == b`
Logical	Combines conditions	`a && b`, `a
Assignment	Assigns values	`a = 10`, `a += 5`
Increment/Decrement	Increases or decreases value	`a++`, `a--`
Bitwise	Works on bits	`a & b`, `a

4. Explain the purpose and use of constants and literals in C++.

Term	Purpose	Example
Constant	A fixed value that cannot be changed during program execution	`const int a = 10;`

Literal	A direct value written in code	`10`, `'A'`, `"Hello"`

3. Control Flow Statements

1. What are conditional statements in C++? Explain the if-else and switch statements.

Conditional Statements in C++

Conditional statements are used to make decisions in a program. They execute different blocks of code based on whether a condition is true or false.

if-else Statement

The **if-else** statement checks a condition.

- If the condition is true, the `if` block is executed.
- If the condition is false, the `else` block is executed.
  It is useful when there are two possible outcomes.

switch Statement

The **switch** statement is used to select one option from many.

- It checks the value of a variable.
- The program executes the matching `case`.
- The `break` statement stops execution after a case.

2. What is the difference between for, while, and do-while loops in

Loop	Description	Condition Check	Example
for	Used when number of iterations is known	Before loop	`for(int i=0;i<5;i++)`
while	Used when condition is checked first	Before loop	`while(i<5)`
do-while	Executes at least once	After loop	`do{ }while(i<5);`

3. How are break and continue statements used in loops? Provide examples.

break and continue Statements in C++

The **break** and **continue** statements are used to control the flow of loops.

break Statement

The **break** statement is used to immediately terminate a loop.
When a `break` is encountered, the program exits the loop and continues with the next statement after the loop.
It is useful when a condition is met and no further iterations are needed.

Example:

```
for(int i=1; i<=5; i++){
    if(i==3)
        break;
    cout<<i<<" ";
}
```

Output: `1 2`

continue Statement

The **continue** statement is used to skip the current iteration of the loop.
When a `continue` is encountered, the program skips the remaining code inside the loop for that iteration and moves to the next iteration.

Example:

```
for(int i=1; i<=5; i++){
    if(i==3)
        continue;
    cout<<i<<" ";
}
```

Output: `1 2 4 5`

4. Explain nested control structures with an example.

Nested Control Structures in C++

Nested control structures mean **using one control statement inside another** (like a loop inside a loop or an if inside another if).

They are used when we need to check multiple conditions or perform repeated tasks within another loop.

Example:

```
#include<iostream>
using namespace std;

int main() {
```

```
    for(int i=1; i<=3; i++) {
        for(int j=1; j<=3; j++) {
            cout << i << j << " ";
        }
        cout << endl;
    }
    return 0;
}
```

4. Functions and Scope

1. What is a function in C++? Explain the concept of function declaration, definition, and calling.

Step	Explanation	Example
Declaration	Tells the compiler about the function (name, return type, parameters)	`int add(int, int);`
Definition	Contains the actual code of the function	`int add(int a, int b){ return a+b; }`
Calling	Invokes (runs) the function in `main()`	`add(2,3);`

2. What is the scope of variables in C++? Differentiate between local and global scope.

Scope of Variables in C++

Scope means the **area of a program where a variable can be used**.

Types of Scope:

Type	Description	Example
Local Scope	Variable declared inside a function; used only within that function	`void fun(){ int x=10; }`
Global Scope	Variable declared outside all functions; can be used anywhere in the program	`int x=10;` (outside main)

3. Explain recursion in C++ with an example.

Recursion in C++

Recursion is a technique where a **function calls itself** to solve a problem.

It must have a **base case** (to stop) and a **recursive case** (to repeat).

Example (Factorial):

```
#include<iostream>
using namespace std;

int fact(int n){
    if(n==0)
        return 1;       // base case
    else
        return n * fact(n-1);   // recursive call
}

int main(){
    cout << fact(5);
    return 0;
}
```

Output:

4. What are function prototypes in C++? Why are they used?

Function Prototypes in C++

A **function prototype** is a declaration of a function that tells the compiler about the function's **name, return type, and parameters** before it is used.

Example:

```
int add(int, int);   // Function prototype
```

Why They Are Used:

- To **inform the compiler** about the function before calling it
- To **avoid errors** during compilation
- To allow functions to be defined **after main()**

5. Arrays and Strings?

1. What are arrays in C++? Explain the difference between single-dimensional and multi- dimensional arrays.

Arrays in C++

An **array** is a collection of elements of the same data type stored in contiguous memory locations.

Types of Arrays:

Type	Description	Example
Single-Dimensional Array	Stores elements in one line (list)	`int a[5] = {1,2,3,4,5};`
Multi-Dimensional Array	Stores elements in rows and columns (table form)	`int a[2][2] = {{1,2},{3,4}};`

2. Explain string handling in C++ with examples.

String Handling in C++

String handling in C++ is used to **store and manipulate text** using the `string` data type (from `<string>` library).

Strings allow operations like **concatenation, length checking, and comparison**.

Common String Operations:

Operation	Description	Example
Input/Output	Read and display string	`cin >> str;, cout << str;`
Concatenation	Join two strings	`s1 + s2`
Length	Find number of characters	`str.length()`
Comparison	Compare two strings	`if(s1 == s2)`

Example:

```
#include<iostream>
#include<string>
using namespace std;

int main() {
    string s1 = "Hello";
    string s2 = "World";

    string s3 = s1 + " " + s2;    // concatenation
    cout << s3 << endl;
    cout << s3.length();          // length

    return 0;
}
```

3. How are arrays initialized in C++? Provide examples of both 1D and 2D arrays.

Array Initialization in C++

Array initialization means **assigning values to an array at the time of declaration**.

1D Array Initialization:

```
int a[5] = {1, 2, 3, 4, 5};
```

or

```
int a[] = {1, 2, 3};
```

2D Array Initialization:

```
int a[2][2] = {
    {1, 2},
    {3, 4}
};
```

or

```
int a[][2] = {
    {5, 6},
    {7, 8}
};
```

4. Explain string operations and functions in C++.

String Operations and Functions in C++

In C++, strings are used to **store and manipulate text** using the `string` class.

Common String Operations:

Operation	Function	Example
Length	`length()`	`str.length();`
Concatenation	`+`	`s1 + s2;`
Access Character	`[]`	`str[0];`
Comparison	`==`	`if(s1 == s2)`
Substring	`substr()`	`str.substr(0,3);`

6. Introduction to Object-Oriented Programming

1. Explain the key concepts of Object-Oriented Programming (OOP).

Key Concepts of Object-Oriented Programming (OOP)

1. **Class**
   A class is a **blueprint** for creating objects.
   Example: `class Student { };`

2. **Object**
   An object is an **instance of a class**.
   Example: `Student s1;`
3. **Encapsulation**
   It means **wrapping data and functions together** in a class and hiding data.
4. **Inheritance**
   It allows one class to **use properties of another class** (code reuse).
5. **Polymorphism**
   It means **one function can have many forms** (same name, different behavior).
6. **Abstraction**
   It means **hiding complex details** and showing only necessary features.

2. What are classes and objects in C++? Provide an example.

Classes and Objects in C++

- **Class**: A class is a **blueprint** used to create objects. It contains data members and functions.
- **Object**: An object is an **instance of a class** that can access its data and functions.

Example:

```
#include<iostream>
using namespace std;

class Student {
public:
    string name;
    void display() {
        cout << name;
    }
};

int main() {
    Student s1;       // object creation
    s1.name = "Raj";
    s1.display();
    return 0;
}
```

3. What is inheritance in C++? Explain with an example.

Inheritance in C++

Inheritance is a feature of OOP where **one class (child/derived class) acquires the properties and functions of another class (parent/base class)**.

It helps in **code reusability** and reduces duplication.

Example:

```
#include<iostream>
using namespace std;

class Animal {
public:
    void sound() {
        cout << "Animal makes sound" << endl;
    }
};

class Dog : public Animal {   // Inheriting Animal class
};

int main() {
    Dog d;
    d.sound();   // using parent class function
    return 0;
}
```

4. What is encapsulation in C++? How is it achieved in classes?

Encapsulation in C++

Encapsulation is the process of **wrapping data and functions together in a single unit (class)** and **restricting direct access to data**.

It helps in **data hiding and security**.

How It Is Achieved:

- By using **classes**
- By declaring data members as **private**
- By accessing them through **public functions (getters/setters)**

Example:

```
#include<iostream>
using namespace std;

class Student {
private:
    int marks;   // hidden data

public:
    void setMarks(int m) {
        marks = m;
```

```
    }
    int getMarks() {
        return marks;
    }
};

int main() {
    Student s;
    s.setMarks(90);
    cout << s.getMarks();
    return 0;
}
```

----------END----------